

UCUENCA

FACULTAD DE INGENIERÍA - DEET

MAESTRÍA EN CIENCIAS DE LA INGENIERÍA ELÉCTRICA

PROYECTO

Detección de Anomalías en Redes

Métricas de Centralidad en Grafos

Análisis de Redes Complejas

Redes Complejas

Cuenca — Ecuador | Abril 2026

Índice

1	Introducción	2
1.1	Objetivos de Aprendizaje	2
1.2	Marco Teórico	2
1.2.1	Representación de Redes como Grafos	2
1.2.2	Métricas de Centralidad	3
1.2.3	Detección de Anomalías con Z-Score	3
1.2.4	Modelo SIR de Propagación de Malware	3
1.2.5	Nodos de Articulación y Puentes	4
1.2.6	Dataset de Trabajo: IoT-23	4
1.2.7	Datasets Alternativos para Análisis de Grafos	4
2	Parte 1: Construcción del Grafo de Red	7
3	Parte 2: Cálculo de Métricas de Centralidad	9
4	Parte 3: Detección de Anomalías Estadísticas	10
5	Parte 4: Simulación de Propagación de Malware (Modelo SIR)	12
6	Parte 5: Resiliencia — Nodos de Articulación y Puentes	14
7	Desafío Extra: Detección de Botnet	16
7.1	Rúbrica de Evaluación	17
A	Referencias	19

Capítulo 1

Introducción

1.1 Objetivos de Aprendizaje

Al finalizar esta práctica, el estudiante será capaz de:

- OA-1. Modelar** una red corporativa como grafo $G = (V, E)$ e implementarla con NetworkX.
- OA-2. Calcular** métricas de centralidad: Degree, Betweenness, Closeness y PageRank.
- OA-3. Detectar** nodos anómalos aplicando z-score sobre las métricas de centralidad.
- OA-4. Simular** la propagación de malware mediante el modelo epidemiológico SIR.
- OA-5. Identificar** nodos de articulación y puentes como activos críticos de infraestructura.
- OA-6. Proponer** estrategias de hardening y redundancia basadas en el análisis de grafos.

Advertencia

Conexión con los contenidos del curso Esta práctica integra los conceptos de los módulos de **Redes Complejas** (grafos, centralidad, detección de comunidades) con aplicaciones directas en **Ciberseguridad**: análisis de botnets, modelado de ataques, propagación de malware en redes Scale-Free y resiliencia de infraestructura crítica.

1.2 Marco Teórico

1.2.1 Representación de Redes como Grafos

Una red de computadoras se modela como un grafo $G = (V, E)$ donde:

- V es el conjunto de **vértices** (hosts, routers, servidores, firewalls)
- $E \subseteq V \times V$ es el conjunto de **aristas** (enlaces de red)
- Cada arista (u, v, w) tiene un peso w que puede representar ancho de banda, latencia o confianza.

1.2.2 Métricas de Centralidad

Las métricas de centralidad cuantifican la *importancia* de cada nodo dentro de la topología de red. Su uso en ciberseguridad permite identificar activos críticos y nodos comprometidos.

Cuadro 1: Métricas de centralidad y su aplicación en ciberseguridad

Métrica	Fórmula	Aplicación en Seguridad
Degree Centrality	$C_d(v) = \frac{\deg(v)}{n-1}$	Detectar hosts con conexiones inusualmente altas (posibles bots).
Betweenness Centrality	$C_b(v) = \sum_{s \neq v \neq t} \frac{\sigma(s, t v)}{\sigma(s, t)}$	Identificar routers críticos y objetivos Man-in-the-Middle.
Closeness Centrality	$C_c(v) = \frac{n-1}{\sum_u d(v, u)}$	Evaluar la velocidad de propagación de malware desde un nodo.
PageRank	$PR(v) = \frac{1-d}{n} + d \sum_{u \in N(v)} \frac{PR(u)}{\deg(u)}$	Priorizar activos de alto valor informacional para proteger.

1.2.3 Detección de Anomalías con Z-Score

Para detectar nodos estadísticamente atípicos se computa un **score compuesto** y se normaliza:

$$\text{score}(v) = 0.5 \cdot C_b(v) + 0.3 \cdot C_d(v) + 0.2 \cdot PR(v) \quad (1.1)$$

$$z(v) = \frac{\text{score}(v) - \mu}{\sigma} \quad (1.2)$$

Un nodo con $z > 1.5$ se clasifica como **anómalo** y requiere investigación adicional.

1.2.4 Modelo SIR de Propagación de Malware

El modelo epidemiológico SIR discreto sobre el grafo G simula la propagación de malware:

$$S(t+1) = S(t) - \beta \cdot k_i \cdot S(t) \cdot I(t) \quad (1.3)$$

$$I(t+1) = I(t) + \beta \cdot k_i \cdot S(t) \cdot I(t) - \gamma \cdot I(t) \quad (1.4)$$

$$R(t+1) = R(t) + \gamma \cdot I(t) \quad (1.5)$$

donde β es la tasa de infección por contacto, γ la tasa de recuperación, y k_i el grado del nodo infectado. El número reproductivo básico es $R_0 = \beta/\gamma$.

i Nota

Condición de epidemia Si $R_0 > 1$, la infección se propaga por la red. En redes Scale-Free, el umbral epidémico tiende a cero ($\beta_c \rightarrow 0$) debido a la heterogeneidad en la distribución de grados.

1.2.5 Nodos de Articulación y Puentes

- **Nodo de articulación** (*cut vertex*): nodo cuya eliminación divide el grafo en componentes desconectados. Su compromiso o fallo aísla segmentos de red.
- **Puente** (*bridge*): arista cuya eliminación aumenta el número de componentes conexos. Representa un *Single Point of Failure* (SPOF) en la infraestructura.

1.2.6 Datasets de Trabajo Recomendados

Para validar las técnicas estudiadas sobre tráfico real, el estudiante debe seleccionar **una** de las siguientes dos capturas, ambas publicadas por el Stratosphere Laboratory de la Universidad Técnica Checa de Praga (CTU) [7]. Las dos opciones son equivalentes en términos de complejidad y permiten cumplir todos los objetivos de la práctica; difieren en la familia de malware analizada y en el tipo de canal C&C.

✓ Recomendación

Opción A — CTU-13: Escenario 10 u 11 (Rbot, C&C IRC)

- **Familia:** Rbot — botnet clásica con **Comando y Control vía IRC**.
- **Capturas sugeridas:** CTU-Malware-Capture-Botnet-51 (escenario 10) o CTU-Malware-Capture-Botnet-52 (escenario 11).
- **Formato:** flujos bidireccionales Argus (*.binetflow) con campos SrcAddr, DstAddr, Sport, Dport, Label.
- **URL:** <https://www.stratosphereips.org/datasets-ctu13>
- **Por qué Rbot:** la topología en estrella alrededor del servidor IRC produce un nodo C&C con Betweenness y Degree extremos — caso ideal para validar la detección por z-score.

✓ Recomendación

Opción B — IoT-23: capture-1-1 (Mirai)

- **Familia:** Mirai — botnet IoT que propaga vía escaneo Telnet/SSH y realiza DDoS coordinado.
- **Captura sugerida:** CTU-IoT-Malware-Capture-1-1 (Mirai infectando una Philips HUE).
- **Formato:** conn.log.labeled de Zeek con etiquetas Malicious + sub-categoría (PartOfAHorizontalPortScan, C&C, DDoS, Okiru, etc.).
- **URL:** <https://www.stratosphereips.org/datasets-iot23>
- **Por qué Mirai:** el patrón de escaneo horizontal masivo genera nodos con

Degree Centrality anómala y comunidades de bots claramente detectables por Louvain.

i Nota

Construcción del grafo desde la captura Independientemente de la opción elegida, el grafo $G = (V, E)$ se construye de la misma manera: cada IP única (**SrcAddr** o **id.orig_h**, **DstAddr** o **id.resp_h**) se convierte en un nodo, y cada par origen-destino observado al menos una vez en los flujos forma una arista. El peso de la arista puede ser el número de flujos, los bytes acumulados o la duración total.

1.2.7 Datasets Alternativos para Análisis de Grafos

Aunque IoT-23 es el dataset principal de la práctica, existen otras fuentes públicas ampliamente utilizadas en la literatura de detección de botnets y anomalías de red. Se clasifican en dos niveles según el grado de preprocesamiento requerido para construir el grafo $G = (V, E)$.

Tier 1 — Listos para construir grafos (NetFlow / Bidirectional Flows)

Estos datasets ya proveen flujos con campos **SrcAddr/DstAddr**, lo que permite generar el grafo de comunicaciones directamente con un **groupby** en pandas o Spark.

Cuadro 2: Datasets Tier 1: NetFlow / flujos bidireccionales etiquetados

Dataset	Origen	Tamaño / Contenido	Ventaja para la práctica
CTU-13	CTU Praga (Stratosphere Lab)	13 escenarios reales: Neriis, Rbot, Virut, Menti, Sogou, Murlo, NSIS-ay. ≈ 1.5 GB de Argus binetflow.	Cada flujo ya tiene SrcAddr→DstAddr — se convierte directo a grafo con groupby .
IoT-23 (Aposemat)	Stratosphere Lab, 2020	23 capturas de IoT infectadas (Mirai, Torii, Hide-and-Seek, Hakai, Okiru, etc.).	Ideal para esta práctica: incluye los IoT-Device del modelo. Labels por flujo.
CIC-IDS2017 / CSE-CIC-IDS2018	Canadian Inst. for Cybersecurity (UNB)	ARES botnet + Zeus + Ares. CSV de 80 features.	El CSV ya tiene Source IP / Destination IP / Flow Duration — conversión a grafo trivial.

Continuación de tabla 2

Dataset	Origen	Tamaño / Contenido	Ventaja para la práctica
Bot-IoT (UNSW)	UNSW Canberra, 2018	72M flujos: DDoS, DoS, recon, theft vía Mirai en testbed con 5 dispositivos IoT.	Etiquetado ataque/normal + categoría + subcategoría.

Tier 2 – PCAPs crudos (requieren preprocesar con Zeek/Argus)

Estos datasets contienen capturas .pcap sin agregar; es necesario procesarlas con Zeek (antes Bro) o Argus para producir flujos antes de construir el grafo.

Cuadro 3: Datasets Tier 2: PCAPs crudos

Dataset	Uso
MCFP — Malware Capture Project	Cientos de capturas de malware real con C&C activo. Contiene la suite original de CTU.
N-BalIoT	Mirai + BASHLITE en 9 IoTs reales (cámaras, monitores de bebé). Features ya extraídas, no requiere Wireshark.
ISOT Botnet	Mezcla Storm + Waledac + tráfico legítimo, formato pcap.

i Nota

Enlaces oficiales

- **CTU-13** → <https://www.stratosphereips.org/datasets-ctu13>
- **IoT-23** → <https://www.stratosphereips.org/datasets-iot23>
- **CIC-IDS2017** → <https://www.unb.ca/cic/datasets/ids-2017.html>
- **CIC-IDS2018** → <https://www.unb.ca/cic/datasets/ids-2018.html>
- **Bot-IoT UNSW** → <https://research.unsw.edu.au/projects/bot-iot-dataset>
- **N-BalIoT** → https://archive.ics.uci.edu/ml/datasets/detection_of_IoT_botnet_attacks_N_BaIoT
- **MCFP capturas** → <https://www.stratosphereips.org/datasets-malware>

Capítulo 2

Parte 1: Construcción del Grafo de Red

i Nota

Escenario Se modela una red corporativa con 20 nodos de cuatro tipos: `firewall`, `router`, `server`, `host/IoT`. La red incluye zonas DMZ, LAN interna y un servidor SIEM para monitoreo.

```
1 import networkx as nx
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import random
5
6 random.seed(42)
7 np.random.seed(42)
8
9 G = nx.Graph()
10
11 # Definicion de nodos: {id: (tipo, nombre)}
12 nodos = {
13     0: ("firewall", "FW-Perimetral"),
14     1: ("router", "Router-Core"),
15     2: ("router", "Router-DMZ"),
16     3: ("server", "Web-Server"),
17     4: ("server", "DB-Server"),
18     5: ("server", "Mail-Server"),
19     6: ("router", "Router-LAN-A"),
20     7: ("router", "Router-LAN-B"),
21     8: ("host", "PC-Admin"),
22     9: ("host", "PC-User1"),
23     10: ("host", "PC-User2"),
24     11: ("host", "PC-User3"),
25     12: ("host", "PC-User4"),
26     13: ("host", "PC-User5"),
27     14: ("host", "Impresora"),
28     15: ("host", "IoT-Device1"),
29     16: ("host", "IoT-Device2"),
30     17: ("server", "SIEM-Server"),
31     18: ("host", "PC-User6"),
32     19: ("host", "PC-User7"),
33 }
34
35 for nid, (tipo, nombre) in nodos.items():
36     G.add_node(nid, tipo=tipo, nombre=nombre)
37
```



```
38 # Aristas: (origen, destino, peso_Mbps)
39 aristas = [
40     (0, 1, 1000), (1, 2, 500), (1, 6, 500), (1, 7, 500),
41     (1, 17, 1000), (2, 3, 100), (2, 5, 100), (6, 4, 100),
42     (6, 8, 100), (6, 9, 100), (6, 10, 100), (6, 14, 10),
43     (7, 11, 100), (7, 12, 100), (7, 13, 100), (7, 15, 10),
44     (7, 16, 10), (7, 18, 100), (7, 19, 100), (8, 4, 100),
45     (3, 5, 100), (17, 3, 100), (17, 5, 100),
46 ]
47
48 for u, v, w in aristas:
49     G.add_edge(u, v, weight=w)
50
51 print(f"Nodos: {G.number_of_nodes()}, Aristas: {G.number_of_edges()}")
52 print(f"Densidad: {nx.density(G):.4f}, Conectado: {nx.is_connected(G)}")
```

Listing 2.1: Construcción del grafo de red corporativa

```
1 Nodos: 20, Aristas: 23
2 Densidad: 0.1211, Conectado: True
```

Listing 2.2: Salida esperada

Preguntas de Análisis

- P1.** ¿Qué representa la *densidad* del grafo en términos de la infraestructura de red? ¿Qué implicaciones tiene para la resiliencia ante ataques?
- P2.** Modifique el código para que el grafo sea **dirigido** (DiGraph). ¿Cómo cambia la interpretación de las métricas de centralidad?

Capítulo 3

Parte 2: Cálculo de Métricas de Centralidad

```
1 degree_c = nx.degree_centrality(G)
2 betweenness_c = nx.betweenness_centrality(G)
3 closeness_c = nx.closeness_centrality(G)
4 pagerank = nx.pagerank(G, alpha=0.85)
5
6 # Mostrar top-5 por Betweenness (mas relevante en seguridad)
7 print(f"\n{'Nodo':>4} {'Nombre':<20} {'Tipo':<10} "
8       f"{'Degree':>8} {'Betw.':>8} {'Close.':>8} {'PR':>8}")
9 print("-" * 70)
10
11 top5 = sorted(betweenness_c.items(), key=lambda x: x[1], reverse=
12               True)[:5]
13 for nid, _ in top5:
14     tipo, nm = nodos[nid]
15     print(f"{'nid':>4} {'nm':<20} {'tipo':<10} "
16           f"{'degree_c[nid]:>8.4f} {'betweenness_c[nid]:>8.4f} "
17           f"{'closeness_c[nid]:>8.4f} {'pagerank[nid]:>8.4f}")
```

Listing 3.1: Cálculo de las cuatro métricas de centralidad

```
1 Nodo Nombre Tipo Degree Betw. Close. PR
2 -----
3 1 Router-Core router 0.2632 0.7154 0.5758 0.2722
4 7 Router-LAN-B router 0.4211 0.6140 0.5135 0.1348
5 6 Router-LAN-A router 0.3158 0.4620 0.4634 0.1018
6 2 Router-DMZ router 0.1579 0.0936 0.4043 0.0341
7 17 SIEM-Server server 0.1579 0.0936 0.4043 0.0900
```

Listing 3.2: Top-5 nodos por Betweenness Centrality

Preguntas de Análisis

- P3.** ¿Por qué el **Router-Core** (ID=1) tiene el mayor Betweenness aunque no es el nodo más conectado? ¿Qué tipo de ataque lo hace un objetivo prioritario?
- P4.** Compare los valores de *Degree Centrality* vs. *Betweenness Centrality*. ¿Siempre coincide el nodo más conectado con el más “intermediario”?
- P5.** Compute el **coeficiente de clustering** global y local. ¿Qué segmentos de la red forman *cliques*?

Capítulo 4

Parte 3: Detección de Anomalías Estadísticas

```
1 # Score compuesto ponderado
2 scores = {}
3 for n in G.nodes():
4     scores[n] = (betweenness_c[n] * 0.5 +
5                 degree_c[n] * 0.3 +
6                 pagerank[n] * 0.2)
7
8 vals = np.array(list(scores.values()))
9 media = np.mean(vals)
10 std = np.std(vals)
11 umbral_z = 1.5
12
13 print(f"Score: mu={media:.4f}, sigma={std:.4f}")
14 print(f"Umbral anomalia: z > {umbral_z} "
15       f"=> score > {media + umbral_z * std:.4f}\n")
16
17 anomalos = {}
18 for n in G.nodes():
19     z = (scores[n] - media) / std if std > 0 else 0
20     if z > umbral_z:
21         tipo, nm = nodos[n]
22         anomalos[n] = z
23         print(f"[ANOMALO] ID={n:>2} | {nm:<20} | {tipo:<10} | z={z:.2f}")
```

Listing 4.1: Score compuesto y detección por z-score

```
1 Score: mu=0.0959, sigma=0.1459
2 Umbral anomalia: z > 1.5 => score > 0.3148
3
4 [ANOMALO] ID= 1 | Router-Core | router | z=2.71
5 [ANOMALO] ID= 7 | Router-LAN-B | router | z=2.50
6 [ANOMALO] ID= 6 | Router-LAN-A | router | z=1.71
```

Listing 4.2: Nodos anómalos detectados

Hallazgo

Nodos críticos detectados Los tres routers de agregación dominan el tráfico de la red. En un entorno real, estos nodos serían objetivos prioritarios para hardening, microsegmentación y monitoreo intensivo en el SIEM.

? Preguntas de Análisis

- P6.** Modifique el umbral a $z > 2.0$. ¿Cuántos nodos se detectan ahora? ¿Cómo balancearía el umbral en un entorno de producción (SIEM)?
- P7.** Añada *Closeness Centrality* al score compuesto con peso 0.1 (redistribuyendo los demás pesos). ¿Cambian los nodos detectados?

Capítulo 5

Parte 4: Simulación de Propagación de Malware (Modelo SIR)

i Nota

Parámetros de simulación

- **Nodo inicial:** IoT-Device1 (ID=15)
- **Pasos:** 20
- β : 0.3 (tasa de infección)
- γ : 0.1 (tasa de recuperación)

$R_0 = \beta/\gamma = 3 \Rightarrow$ Condición de epidemia ($R_0 > 1$).

```
1 def simular_sir(G, nodo_inicial, beta=0.3, gamma=0.1, pasos=20):
2     """Modelo SIR discreto sobre grafo. Retorna historiales S, I, R.
3     """
4     estado = {n: 'S' for n in G.nodes()}
5     estado[nodo_inicial] = 'I'
6     hS, hI, hR = [], [], []
7
8     for paso in range(pasos):
9         nuevo_estado = estado.copy()
10        for nodo in G.nodes():
11            if estado[nodo] == 'I':
12                # Intentar infectar vecinos susceptibles
13                for vecino in G.neighbors(nodo):
14                    if estado[vecino] == 'S' and random.random() < beta:
15                        nuevo_estado[vecino] = 'I'
16                # Recuperacion espontanea
17                if random.random() < gamma:
18                    nuevo_estado[nodo] = 'R'
19        estado = nuevo_estado
20        s = sum(v == 'S' for v in estado.values())
21        i = sum(v == 'I' for v in estado.values())
22        r = sum(v == 'R' for v in estado.values())
23        hS.append(s); hI.append(i); hR.append(r)
24
25    return hS, hI, hR, estado
26
27 # Ejecutar simulacion desde IoT-Device1
28 hS, hI, hR, estado_final = simular_sir(G, nodo_inicial=15)
```

```
29 # Visualizar curvas SIR
30 fig, ax = plt.subplots(figsize=(8, 4))
31 pasos = range(1, len(hS) + 1)
32 ax.plot(pasos, hS, color='#2196F3', lw=2, label='Susceptibles (S)')
33 ax.plot(pasos, hI, color='#A51008', lw=2, label='Infectados (I)')
34 ax.plot(pasos, hR, color='#4CAF50', lw=2, label='Recuperados (R)')
35 ax.fill_between(pasos, hI, alpha=0.15, color='#A51008')
36 ax.set_xlabel('Paso de tiempo'); ax.set_ylabel('Nodos')
37 ax.set_title('Propagacion de Malware - Modelo SIR (origen: IoT-
    Device1)')
38 ax.legend(); ax.grid(alpha=0.3)
39 plt.tight_layout()
40 plt.savefig('sir_propagacion.png', dpi=150)
```

Listing 5.1: Implementación del modelo SIR sobre el grafo

Preguntas de Análisis

- P8.** Ejecute la simulación cambiando el nodo inicial a **Router-Core** (ID=1). Compare la tasa de ataque final $|R|/n$ con la simulación desde IoT-Device1. ¿Por qué difieren?
- P9.** Experimente con $\beta \in \{0.1, 0.2, 0.3, 0.5\}$ manteniendo $\gamma = 0.1$. ¿Para qué valor de β se tiene $R_0 < 1$ y la infección se extingue?
- P10.** Implemente una estrategia de **cuarentena**: después del paso 5, elimine las aristas del nodo más infectado. ¿Cómo cambian las curvas SIR?

Capítulo 6

Parte 5: Resiliencia — Nodos de Articulación y Puentes

```
1 # Nodos de articulacion (cut vertices)
2 articulation_pts = list(nx.articulation_points(G))
3 print(f"Nodos de articulacion ({len(articulation_pts)} encontrados)
4 :")
5 for n in articulation_pts:
6     tipo, nm = nodos[n]
7     print(f" ID={n:>2} | {nm:<20} | Grado={G.degree(n)}")
8
9 # Puentes (Single Points of Failure)
10 bridges = list(nx.bridges(G))
11 print(f"\nPuentes criticos ({len(bridges)} encontrados):")
12 for u, v in bridges:
13     print(f" {nodos[u][1]} <--> {nodos[v][1]}")
14
15 # Analisis de impacto: eliminar Router-Core
16 G_test = G.copy()
17 G_test.remove_node(1) # Eliminar Router-Core
18 componentes = list(nx.connected_components(G_test))
19 print(f"\nSin Router-Core: {len(componentes)} componentes")
20 print(f"Tamanos: {sorted([len(c) for c in componentes], reverse=
    True)}")
```

Listing 6.1: Identificación de activos críticos de infraestructura

```
1 Nodos de articulacion (3 encontrados):
2   ID= 1 | Router-Core | Grado=4
3   ID= 6 | Router-LAN-A | Grado=6
4   ID= 7 | Router-LAN-B | Grado=8
5
6 Puentes criticos (13 encontrados):
7   FW-Perimetral <--> Router-Core
8   Router-Core <--> Router-LAN-A
9   Router-Core <--> Router-LAN-B
10   ... (10 puentes adicionales a hosts/IoT)
11
12 Sin Router-Core: 3 componentes
13 Tamanos: [9, 7, 2]
```

Listing 6.2: Resultados de resiliencia

Preguntas de Análisis

P11. La eliminación de **Router-Core** divide la red en 3 componentes. ¿Qué hosts quedan aislados? ¿Cómo afecta esto a la operación del negocio?

P12. Proponga las **aristas mínimas** a añadir para eliminar todos los puentes y garantizar 2-conectividad en la red. Implemente los cambios y verifique con `nx.is_2_edge_connected(G)`.

Capítulo 7

Desafío Extra: Detección de Botnet

⚠ Advertencia

Escenario avanzado Una botnet se caracteriza por un conjunto de nodos que se comunican densamente entre sí (comando y control C&C) pero tienen pocas conexiones hacia el resto de la red. En grafos, esto genera una **comunidad** con alta modularidad interna y baja centralidad de intermediación hacia el exterior.

```
1 # Agregar nodos botnet al grafo existente
2 G_botnet = G.copy()
3 bot_ids = list(range(20, 25)) # 5 nodos bot
4
5 for bid in bot_ids:
6     G_botnet.add_node(bid, tipo='bot', nombre=f'Bot-{bid-19}')
7
8 # Conexiones internas densas (C&C)
9 for i in bot_ids:
10     for j in bot_ids:
11         if i < j:
12             G_botnet.add_edge(i, j, weight=100)
13
14 # Pocas conexiones externas (canal de exfiltracion)
15 G_botnet.add_edge(bot_ids[0], 9, weight=1) # Bot maestro -> PC-
    User1
16 G_botnet.add_edge(bot_ids[0], 12, weight=1) # Bot maestro -> PC-
    User4
17
18 # Recalcular metricas
19 bc_botnet = nx.betweenness_centrality(G_botnet)
20 cc_botnet = nx.average_clustering(G_botnet, nodes=bot_ids)
21 print(f"Clustering interno botnet: {cc_botnet:.4f}") # Cercano a
    1.0
22 print(f"Clustering red normal: "
23       f"{nx.average_clustering(G_botnet, nodes=list(range(20))):.4f}
24       ")
25
26 # Deteccion por algoritmo Louvain (comunidades)
27 try:
28     from networkx.algorithms.community import louvain_communities
29     comunidades = louvain_communities(G_botnet, seed=42)
30     for i, com in enumerate(comunidades):
31         bots_en_com = com & set(bot_ids)
32         if bots_en_com:
33             print(f"Comunidad {i}: contiene bots {bots_en_com}")
```

```

33 except ImportError:
34     print("Instalar: pip install networkx[default]")

```

Listing 7.1: Simulación e identificación de una botnet

🔍 Preguntas de Análisis

- P13.** ¿Qué métrica distingue mejor a los nodos bot del resto? ¿Degree Centrality, Betweenness o Clustering Coefficient? Justifique.
- P14.** El algoritmo de Louvain detecta comunidades con alta modularidad. ¿Por qué los nodos bot forman una comunidad separada?
- P15.** Diseñe un sistema de alertas que combine **tres umbrales** (clustering > 0.8, betweenness externo < 0.01, degree externo < 2) para detectar botnets automáticamente.

7.1 Rúbrica de Evaluación

Cuadro 4: Rúbrica de evaluación de la práctica

Criterio		Excelente (10)	Satisfactorio (7)	Mínimo (5)	Peso
Construcción del grafo		G correcto con todos los atributos y visualización	G correcto sin atributos	G incompleto	15%
Métricas de centralidad		4 métricas + análisis comparativo completo	3 métricas correctas	2 métricas	20%
Detección de anomalías		Z-score + justificación del umbral	Z-score correcto	Identifica anomalías	20%
Simulación SIR		Código + análisis de β + gráficas	Código funcional	Intento de simulación	20%
Resiliencia		Articulación + propuesta de mejoras	Identifica correctamente	Lista parcial	15%
Respuestas escritas		Todas las preguntas + reflexión crítica	8 de 13 preguntas	5 de 13 preguntas	10%

Formato de Entrega

1. **Archivo Python:** `practica_redes_[apellido].py` con código completamente comentado.
2. **Reporte PDF:** Documento con respuestas justificadas a las 13 preguntas (mínimo 3 párrafos por pregunta).
3. **Visualizaciones:** Capturas de pantalla o figuras exportadas de los seis paneles generados por matplotlib.
4. **Plazo:** Una semana desde la sesión de laboratorio.

Capítulo A

Referencias

- [1] Barabási, A.-L. (2016). *Network Science*. Cambridge University Press. <http://networksciencebook.com>
- [2] Newman, M. E. J. (2010). *Networks: An Introduction*. Oxford University Press.
- [3] Hagberg, A., Swart, P., & Schult, D. (2008). Exploring network structure, dynamics, and function using NetworkX. *Proc. 7th Python in Science Conference (SciPy2008)*, 11–15.
- [4] Chakrabarti, D., Wang, Y., Wang, C., Leskovic, J., & Faloutsos, C. (2008). Epidemic thresholds in real networks. *ACM TISSEC*, 10(4), 1–26.
- [5] Strogatz, S. H. (2001). Exploring complex networks. *Nature*, 410, 268–276.
- [6] Boccaletti, S., et al. (2006). Complex networks: Structure and dynamics. *Physics Reports*, 424(4–5), 175–308.
- [7] García, S., Parmisano, A., & Erquiaga, M. J. (2020). *IoT-23: A labeled dataset with malicious and benign IoT network traffic* (Version 1.0.0) [Data set]. Stratosphere Laboratory, CTU University, Prague. <https://www.stratosphereips.org/datasets-iot23>